

UNIVERSITY OF JOHANNESBURG

Blockchain Assignment 9

COLLEGE OF BUSINESS AND ECONOMICS

MASTERS IN FINANCIAL ENGINEERING

Financial Blockchain Capstone: African Remittances Payment Prototype

---

Author:	*Student Number:*
Siphosethu Limane	226214675
Keitumetse Motlhabane	220068138
Masixole Ntshangase	226272432
Joelle Behoor	226259509

1 EXECUTIVE SUMMARY

Our project demonstrates a simple payment platform using a simulated stablecoin called MiniRandCoin, or mZAR. A sender deposits simulated local currency, receives mZAR, and sends some of it to a recipient. The recipient then exchanges the received mZAR for simulated local currency.

The platform uses a one-to-one reserve rule. Each mZAR in circulation must be matched by one local currency unit in the reserve. Deposits increase the reserve and token supply. Transfers move tokens between users without changing the reserve. Redemptions reduce both the reserve and token supply.

The demonstration starts with a deposit of 5,000 local currency units. The sender transfers 3500 mZAR to the recipient, and the transfer is included in a mined block. The recipient redeems 3500 mZAR. At the end, the sender holds 1500 mZAR, the recipient holds zero, and the reserve contains 1500 local currency units. The code also rejects a changed transaction and a transfer that exceeds the sender's balance.

This project helps explain the separate roles of token accounting, payment records and reserve custody. A blockchain record can show that a transfer was included, but it cannot prove that real money exists in a bank account. This matters for remittances because recording a token transfer is only one step, the recipient must also be able to redeem the tokens into usable local currency.

The current prototype is a Python notebook with a Flask payment API. It uses toy wallets and signatures, stores data in memory, and does not connect to banks or identity-checking services. The API exposes deposits, transfers, mining, redemption, balances and explorer information. Transactions created through `build_transfer()` include transaction IDs. Persistent storage and a complete deposit-event record remain areas for improvement.

2 ARCHITECTURE

2.1 Main Components

The platform runs in one Python process. Its main components are wallets, the reserve ledger, transaction functions, mining functions, verification functions and an explorer. The function `run_remittance_demo()` connects these components for the main demonstration.

Figure 2.1 shows the main payment flow.

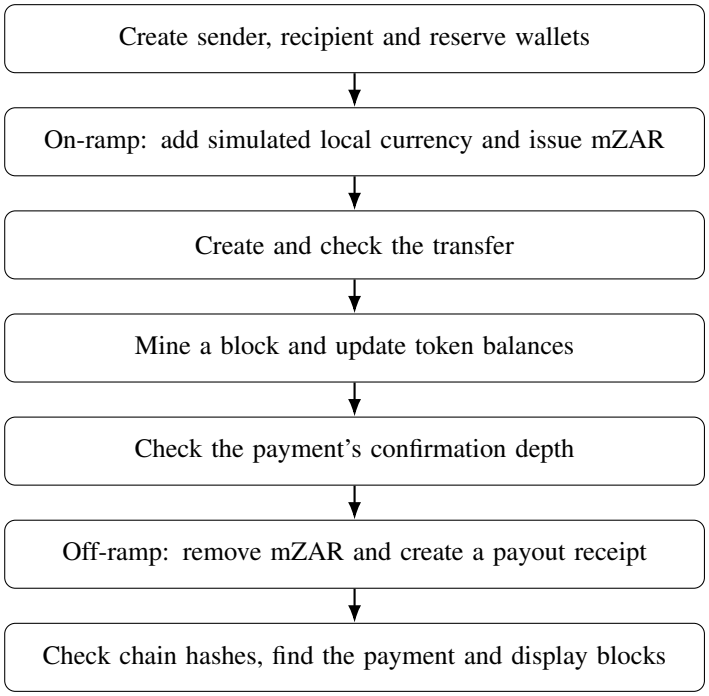


Figure 2.1: The main payment flow in the notebook.

2.2 Wallets and Transaction Checks

The `Wallet` class stores a label, a toy secret key, a toy public key and an address. The function `create_wallets()` creates wallets for the sender, recipient and reserve agent.

The keys are made by hashing fixed labels. This means that the same labels produce the same wallets each time. This is useful for a repeatable demonstration, but it is unsafe for real funds.

Each transfer includes the sender, recipient, amount, timestamp, asset name and signature. The code puts this information into a consistent JSON format before hashing it.

The signature is a toy hash made from the secret key and the transaction data. Verification repeats this calculation using the same secret key. It can detect a transaction that was changed after signing. However, it is not a real digital signature that another person can check using only a public key.

2.3 Reserve Ledger and Data Model

The `ReserveLedger` class stores the financial state of the platform.

Field	Meaning
`vault_fiat`	Simulated local currency held in reserve.
`circulating`	Total mZAR currently in circulation.
`balances`	A dictionary linking each wallet address to its mZAR balance.
`receipts`	A list of simulated local-currency payout receipts.

Table 2.1: The reserve ledger's main fields.

The reserve rule is:

$$\texttt{vault_fiat} = \texttt{circulating}.$$

The function `invariant_ok()` checks this equality.

During an on-ramp, `onramp()` increases the reserve, token supply and sender's balance by the deposit amount. During a transfer, `apply_transfer()` reduces the sender's balance and increases the recipient's balance. The reserve and supply stay the same.

During an off-ramp, `offramp()` reduces the recipient's balance, token supply and reserve. It also creates a receipt with a receipt identifier, wallet address and payout amount.

The expected demonstration balances are shown below.

The reserve check has a limited role. It compares reserve value with supply, but does not check that all wallet balances add up to the supply or that every balance is non-negative. Deposit events are not stored in a separate event log. Redemption receipts are stored in memory and are not included in the blockchain.

Stage	Reserve	Supply	Sender	Recipient
Start	0	0	0	0
Deposit 5,000	5,000	5,000	5,000	0
Transfer 3500	5,000	5,000	1500	3500
Redeem 3500	1500	1500	1500	0

Table 2.2: Reserve and token balances during the demonstration.

2.4 Mining and Balance Updates

Each block contains an index, timestamp, transaction list, previous-block hash, nonce and block hash.

Mining uses SHA-256 with a difficulty of two leading hexadecimal zeroes. The miner changes the nonce until it finds a suitable hash. This low difficulty allows blocks to be mined quickly during the demonstration.

The notebook shows two transfer paths. In the mempool path, `accept_tx()` checks the toy signature, positive amount and available balance. Accepted transfers wait in the mempool. The function `mine_pending()` mines them and updates balances.

The main end-to-end demonstration checks its transfer and mines it directly, without using the mempool. The mempool does not reserve money for pending transfers. As a result, several pending transfers could pass separate balance checks but together exceed the sender's balance. The code also lacks a rollback mechanism if a balance update fails during processing.

2.5 Confirmations and Verification

The code calculates confirmation depth as:

$$k = H - h + 1,$$

where H is the latest block height and h is the height of the block containing the payment.

The main demonstration uses one confirmation by default. Another notebook cell shows two confirmations by mining an extra block. The calling code checks confirmation depth before redemption; `offramp()` does not enforce this check itself.

The function `verify_chain()` checks that block hashes are correct and that each block links to the previous one. It does not check the mining difficulty, transaction signatures or the full balance history.

The function `find_tx_block_index()` searches for a matching transaction. In the step-by-step demonstration, it finds the payment using its transaction ID. The explorer displays block numbers, block hashes, transaction IDs, sender and recipient addresses, and amounts. Transactions created through `build_transfer()` contain an ID. However, the separate `run_remittance_demo()` function still builds its transaction manually without an ID, so this feature is not yet used consistently across all paths.

2.6 Payment API

The prototype includes a Flask payment API that connects the reserve ledger, wallets, mempool, mining functions and explorer. It provides endpoints for deposits, transfers, mining, redemption, balance checks and blockchain inspection.

The main endpoints are:

- `POST /onramp`: adds simulated reserves and issues matching mZAR.
- `POST /transfer`: checks a submitted transfer and adds it to the mempool.
- `POST /mine`: mines pending transactions and updates balances.
- `POST /offramp`: redeems mZAR and returns a simulated payout receipt.
- `GET /balances/<address>`: returns the balance of an address and reserve information.
- `GET /chain`: returns the blockchain and the result of its hash-and-link checks.
- `GET /explorer`: returns block and transaction information.
- `POST /remit`: runs the full simulated payment flow after resetting the API's existing state.

The notebook includes tests using Flask's `test_client()`. These exercise the endpoints inside Python without starting a listening web server. The API uses its own in-memory ledger, chain and mempool, separate from the step-by-step notebook demonstration.

The `/remit` endpoint creates its transaction through `build_transfer()`, so its explorer output includes a transaction ID. It is a demonstration endpoint: it clears previous API records and does not enforce the notebook's confirmation-depth policy.

3

PART B: ERC-20 AND THE ACCOUNT MODEL

3.1 State Representation

Ethereum account model. Ethereum uses accounts to represent its state. An externally owned account is controlled through a private key, while a contract account is controlled by its code. Transactions can change state and trigger contract execution (Antonopoulos and Wood, 2018, Chapter 2, "Externally Owned Accounts (EOAs) and Contracts", p. 64; Chapter 6, "Transactions", p. 159). Native ETH balances are handled by the protocol, whereas ERC-20 balances are stored within the token contract (Antonopoulos and Wood, 2018, Chapter 10, "Tokens on Ethereum", pp. 321–322). Our prototype uses a simpler account model: a Python dictionary connects wallet addresses to their current mZAR balances. Ethereum page references in this chapter follow the printed pagination of the ebook.

Bitcoin UTXO model. Bitcoin represents spendable value through unspent transaction outputs, called UTXOs. A transaction consumes existing outputs and creates new ones. An output must be spent in full, so remaining value is returned through a change output. A wallet calculates its balance from the outputs it can spend (Antonopoulos, 2015, Chapter 5, "Transaction Outputs and Inputs" and "Transaction Outputs", PDF pp. 136–138). Our prototype does not select or consume outputs. Instead, it directly reduces the sender's balance and increases the recipient's balance.

3.2 ERC-20 Functions and Data Structures

ERC-20 defines a common interface for fungible tokens. Fungible means that units of the same token are interchangeable (Antonopoulos and Wood, 2018, Chapter 10, "Tokens and Fungibility", p. 317, and "The ERC20 Token Standard", p. 322).

The functions most relevant to our prototype are:

- balanceOf(address) returns the token balance of an address.
- transfer(recipient, amount) moves tokens from the caller's balance to the recipient.
- approve(spender, amount) authorises another address to spend up to an allowance.

- transferFrom(sender, recipient, amount) allows an authorised spender to transfer tokens within that allowance.

These functions are described in Chapter 10, "ERC20 required functions and events", pp. 322–323 (Antonopoulos and Wood, 2018, pp. 322–323).

ERC-20 contracts track balances and allowances using mappings. The balance mapping records token ownership, while the allowance mapping records delegated spending permissions (Antonopoulos and Wood, 2018, Chapter 10, "ERC20 data structures", pp. 324–325). Our `balances` dictionary provides a simple analogue of the balance mapping, but the prototype has no allowance mapping.

The textbook distinguishes direct transfers from the two-step `approve` and `transferFrom` workflow (Antonopoulos and Wood, 2018, Chapter 10, "ERC20 workflows: transfer and approve & transferFrom", pp. 325–326). Our capstone demonstrates direct payments without delegated spending.

3.3 Comparison Table

Area	Ethereum / ERC-20	Our Capstone
State representation	Ethereum uses account state. Bitcoin instead uses spendable UTXOs.	Uses current balances stored against wallet addresses.
Token balances	Stored in the token contract, commonly through an address-to-balance mapping.	Stored in the Python dictionary `ReserveLedger.balances`.
Supply control	Supply may be fixed or variable. Minting and burning authority depends on the contract implementation.	`onramp()` increases supply; `offramp()` reduces it. Reserve-agent permissions are not enforced.
Verification surface	Full nodes independently validate. Remote clients depend on providers. Light clients use authenticated headers and proofs.	`verify_chain()` checks local block hashes and links. Lookup searches stored transactions.
Transfer interface	Provides `transfer`, `balanceOf`, `approve` and `transferFrom`.	Provides Python functions and Flask routes, but no ERC-20 allowance mechanism.
Authorisation	Transactions use public-key digital signatures.	Toy signatures are checked using the sender's secret key.

Table 3.1: Comparison between ERC-20 and the classroom capstone.

Table 3.1 draws on the textbook's discussion of token interfaces and mappings (Antonopoulos and Wood, 2018, Chapter 10, pp. 322–325), client verification (Antonopoulos and Wood, 2018, Chapter 3, "Should I Run a Full Node?", pp. 86–87), and digital signatures (Antonopoulos and Wood, 2018, Chapter 6, "How Digital Signatures Work", p. 180, and "Verifying the Signature", p. 182).

3.4 How Our Capstone Differs from a Public ERC-20 Stablecoin

Our capstone stores mZAR balances in a Python dictionary and records transfers on a local blockchain. An ERC-20 stablecoin stores token balances in a smart contract, whose rules are executed by network nodes (Antonopoulos and Wood, 2018, Chapter 10, "Tokens on Ethereum", pp. 321–322). Our program runs in one process, where the operator controls the code and data. Its Flask API exposes payment operations, but does not make the system decentralised or ERC-20 compatible.

The main financial difference is reserve enforcement. Our deposit function increases simulated backing and token supply together. Redemption reduces both, while transfers leave the totals unchanged. This demonstrates the one-to-one rule, but equal numbers do not prove that real money exists or is available for payout. The textbook explains that external assets remain dependent on custodians fulfilling their obligations (Antonopoulos and Wood, 2018, Chapter 10, "Counterparty Risk", pp. 317–318). A public fiat-backed stablecoin therefore still needs reliable reserve and redemption arrangements.

Supply authority is simplified too. Our Python methods issue and redeem tokens without checking reserve-agent permissions. A public token can restrict these actions through contract access controls. However, controlled minting does not independently prove that the issuer received a matching bank deposit. Token supply management and verification of external reserves are separate responsibilities.

Our prototype performs no Know Your Customer checks, or KYC. Sender and recipient labels are not verified identities. In a fuller system, customer checks would sit at registration, deposit and redemption boundaries. Personal documents should remain outside the public ledger. Transfer restrictions, if required, would need additional controls. A valid address or signature does not establish a customer's identity or eligibility for payout.

The confirmation policy is a classroom example. The notebook counts the payment block as one confirmation and illustrates two by mining another block. One process controls the entire chain, so this does not model independent validators or reorganisations. A public-chain service would need a payout policy suited to its network and payment risk. Our redemption method and API routes do not enforce confirmation depth internally.

Custody and verification also differ. Real signatures can be checked using a public key (Antonopoulos and Wood, 2018, Chapter 6, "How Digital Signatures Work", p. 180, and "Verifying the Signature", p. 182). Our toy scheme requires the secret key. Our chain checker only checks hashes and links. The textbook distinguishes independent validation from trusting a remote client (Antonopoulos and Wood, 2018, Chapter 3, "Should I Run a Full Node?", pp. 86–87).

We chose the semester stack because each payment step is easy to inspect in Python. The API exposes

operations, and transactions created through `build_transfer()` include identifiers. However, persistent storage, complete deposit-event records and consistent validation remain unfinished. The prototype demonstrates ledger, custody and verification concepts without claiming production-level security or settlement.

4 STABLECOIN IMPLEMENTATION

Chapter 2 described the main components of the platform. This chapter shows how those components work together by following one remittance payment through the notebook, from the first deposit to the final payout. The same example is used throughout: the sender deposits 5,000 local currency units, sends 3500 mZAR to the recipient, and the recipient redeems the full 3500 mZAR.

4.1 Implementation Settings

The implementation is written in Python and runs in a single notebook process. All data is stored in memory. The mining difficulty is deliberately low so that blocks can be mined in a few seconds during a classroom demonstration. The main settings and functions are summarised in Table 4.1.

Function or setting	Role in the implementation
`DIFFICULTY = 2`	Number of leading hexadecimal zeroes needed for a valid block hash.
`create_wallets()`	Creates the sender, recipient and reserve agent wallets.
`onramp()`	Adds simulated local currency to the reserve and issues mZAR.
`build_transfer()`	Creates a signed transfer with a transaction ID.
`accept_tx()`	Checks a transfer and adds it to the mempool.
`mine_pending()`	Mines pending transfers into a block and updates balances.
`apply_transfer()`	Moves mZAR from the sender to the recipient.
`offramp()`	Removes mZAR, reduces the reserve and creates a payout receipt.
`invariant_ok()`	Checks that the reserve equals the circulating supply.
`verify_chain()`	Checks block hashes and links between blocks.
`run_remittance_demo()`	Runs the full payment flow and returns the results.

Table 4.1: Main functions and settings used in the implementation.

4.2 Notation

To describe how each step changes the platform's state, we use the following notation:

- V: simulated local currency held in the reserve (`vault_fiat`);
- S: total mZAR in circulation (`circulating`);
- B_s and B_r: the mZAR balances of the sender and recipient.

The reserve rule from Section 2.3 is

$$V = S. \tag{4.1}$$

The implementation checks this rule after each main step using `invariant_ok()`.

4.3 Wallet Setup

The notebook first calls `create_wallets()` to create three wallets: one for the sender, one for the recipient and one for the reserve agent. Each wallet has a toy secret key, a toy public key and an address. These values are produced by hashing fixed labels with SHA-256, so the same wallets are created every time the notebook runs. This makes the demonstration repeatable, but it would not be safe for real funds.

4.4 On-Ramp: Issuing mZAR

During the on-ramp, the sender deposits d units of simulated local currency. The function `onramp()` then updates the ledger as follows:

$$V \leftarrow V + d, \qquad S \leftarrow S + d, \qquad B_s \leftarrow B_s + d. \tag{4.2}$$

With $d = 5{,}000$, the reserve, the circulating supply and the sender's balance all become 5,000. The reserve and the supply rise by the same amount, so Equation (4.1) still holds.

4.5 Creating and Checking the Transfer

The sender then creates a transfer of $a = 3500$ mZAR. The transaction stores the sender address, recipient address, amount, timestamp and asset name. The code puts these fields into a consistent JSON format, hashes them, and adds a toy signature made from the sender's secret key and the transaction data. When the transfer is created through `build_transfer()`, it also receives a transaction ID, which is later used to find the payment on the chain. Before a transfer is accepted, the code checks that:

1. the signature matches the transaction data;

2. the amount is positive; and

3. the sender has enough mZAR.

The signature check shows whether the transaction was changed after signing. Because the same secret key is used to check it, it does not prove who the sender is the way a real public-key signature would.

4.6 Mining the Payment

The notebook shows two ways to include a transfer in a block. In the mempool path, `accept_tx()` adds valid transfers to the mempool and `mine_pending()` later mines them into a block. In the main end-to-end demonstration, the transfer is checked and mined directly. In both cases, the miner builds a block containing the index, timestamp, transactions and previous-block hash. It then changes the nonce until the block's SHA-256 hash starts with two zeroes. Once the block is added, `apply_transfer()` updates the balances:

$$B_s \leftarrow B_s - a, \qquad B_r \leftarrow B_r + a. \tag{4.3}$$

After the 3500 mZAR transfer, the sender holds 1500 mZAR and the recipient holds 3500 mZAR. The values of V and S stay at 5,000, because a transfer changes who owns the tokens but does not create or destroy any.

4.7 Confirmation Depth

Before the payout, the calling code checks how many blocks confirm the payment. The confirmation depth is

$$k = H - h + 1, \tag{4.4}$$

where H is the height of the latest block and h is the height of the block containing the payment. The payment is in block 1, so it has one confirmation as soon as that block is mined. In a separate cell, one extra block is mined, which raises the depth to $k = 2$. The check is done before `offramp()` is called and is not built into the redemption function.

4.8 Off-Ramp: Redeeming mZAR

The recipient then redeems $r = 3500$ mZAR. The function `offramp()` reverses the on-ramp:

$$B_r \leftarrow B_r - r, \qquad S \leftarrow S - r, \qquad V \leftarrow V - r. \tag{4.5}$$

It also creates a payout receipt containing a receipt identifier, the recipient's address and the amount paid out. After redemption, the recipient holds 0 mZAR, the sender still holds 1500 mZAR, and both V and S equal 1500. The reserve rule still holds. The full sequence of balances matches table seen in Chapter 2.

4.9 Rejecting Invalid Transactions

The implementation tests two invalid cases:

1. **Tampered transaction:** the amount is changed after signing. The recalculated signature no longer matches, so the transfer is rejected.
2. **Insufficient funds:** the sender tries to send 5,000 mZAR while holding only 5,000 mZAR. The balance check fails, so the transfer is rejected.

In both cases, the transaction is stopped before it reaches the mempool or a block.

4.10 Explorer and Chain Verification

At the end of the run, `verify_chain()` recalculates each block hash and checks that every block points to the correct previous block. In the step-by-step demonstration, `find_tx_block_index()` finds the payment using its transaction ID. The explorer shows each block's number and hash, together with the transaction ID, sender address, recipient address and amount. This gives a simple audit trail of the payment. Because `run_remittance_demo()` builds its transaction manually, transactions created on that path do not yet carry an ID (Section 2.6). The verification step checks hashes and links only. It does not recheck the mining difficulty, signatures or the full balance history.

4.11 End-to-End Flow

The function `run_remittance_demo()` connects all of the steps above in the following order:

1. create the sender, recipient and reserve wallets;
2. deposit 5,000 local currency units and issue 5,000 mZAR;

3. create, sign and check a 3500 mZAR transfer;
4. mine the transfer into a block and update the balances;
5. check the confirmation depth;
6. redeem 3500 mZAR and create a payout receipt;
7. check the reserve rule and verify the chain;
8. confirm that invalid transfers are rejected.

The same flow is also available through the Flask `/remit` endpoint described in Section 2.6. That endpoint uses its own in-memory ledger and chain, clears earlier records, and does not apply the notebook's confirmation-depth check. The function returns the final balances, reserve values, receipt, verification results and explorer data in a single structured result. The final state is shown in Table 4.2.

Item	**Final value**
Reserve (V)	1500
Circulating supply (S)	1500
Sender balance (B_s)	1500 mZAR
Recipient balance (B_r)	0 mZAR
Reserve rule satisfied	True
Chain verification passed	True

Table 4.2: Final state after the end-to-end demonstration.

These results show that the implementation keeps the reserve and the token supply equal through the deposit, transfer and redemption steps. The wallet balances are also consistent with the supply, since $B_s + B_r = 1500 + 0 = 1500 = S$, although the code does not check this automatically. The detailed outputs from the test run are presented in Chapter 6, and the main limitations of this classroom implementation are discussed in Chapter 7.

5 FINANCIAL DESIGN

5.1 Financial Design

Reserve Rule and Redemption

The prototype used a 1:1 reserve rule for MiniRandCoin (mZAR), under which each unit of stablecoin issued was matched by one unit of local currency recorded in the reserve ledger. The implementation maintained the invariant

$$\texttt{vault_fiat} = \texttt{circulating}.$$

The invariant was enforced across the three stages of the payment process. During the on-ramp, a local currency deposit increased the recorded reserve and an equivalent amount of mZAR was issued to the sender. A transfer between wallets changed the distribution of mZAR but did not change the reserve balance or the total amount in circulation. During the off-ramp, the recipient's mZAR was redeemed, the circulating supply and recorded reserve were reduced by the redemption amount, and a payout receipt was generated.

The demonstration used an initial deposit of 5,000 local currency units, followed by a transfer and redemption of 3500 mZAR. After redemption, the sender retained 1500 mZAR, while the recipient's balance was zero and the recorded reserve was 1500 local currency units. The reserve invariant remained satisfied. This result demonstrated consistency within the prototype's accounting rules. It did not establish that an external reserve account held sufficient funds, since the reserve was simulated rather than independently verified.

Financial and Operational Risks

Depeg risk: The 1:1 rule was an accounting assumption within the prototype, not a guarantee of market value or redemption under all circumstances. A stablecoin may trade below its intended value if users doubt the adequacy or accessibility of its reserves, or if redemption cannot be completed when requested. Consequently, a production system would need to distinguish between the number of tokens recorded in circulation and the availability of sufficient, accessible local currency to meet

redemption requests. Independent reserve verification and reliable redemption arrangements would be necessary to support confidence in the stated peg. These safeguards were outside the scope of the simulated reserve ledger.

Custody risk: The ability to authorise a transfer depends on control of the relevant private key. Loss, theft or unauthorised use of a key could therefore compromise access to the associated funds. The prototype used simplified wallet functionality to demonstrate transaction signing and verification; it was not a production custody arrangement. A deployed service would need to establish how keys are generated, stored, protected and recovered, and who is responsible for losses arising from compromised credentials. The integrity of the blockchain record would not, by itself, prevent these custody failures.

Anti-Money Laundering (AML) and Know Your Customer(KYC) considerations

The prototype demonstrated the movement and verification of value, rather than a complete regulated remittance process. It did not establish the identity of customers or implement a comprehensive anti-money laundering (AML) and know-your-customer (KYC) framework. A real service would need to determine how customers are identified, how transactions are monitored for suspicious activity, and how applicable reporting and record-keeping obligations are met. These controls would need to operate alongside transaction-signature and chain-integrity checks, since a cryptographically valid transaction does not establish that the transaction complies with financial regulation.

When a Public Blockchain May Not Be Appropriate

A public blockchain should not be selected solely because it provides verifiable transaction records. Its suitability depends on the requirements of the payment service, including confidentiality, transaction costs, processing performance, governance and the applicable regulatory environment. A public ledger may be unsuitable where transaction details could expose sensitive financial information, where predictable processing costs and control over transaction handling are essential, or where the operator requires permissioned participation and clearly defined governance.

For a remittance service, the choice of ledger must also be considered separately from the reserve and redemption arrangements. A verifiable on-chain transfer does not guarantee that a recipient can obtain local currency, that the reserve is adequate, or that the payment satisfies applicable compliance requirements. A permissioned ledger or conventional payment infrastructure may therefore be more appropriate where controlled access, confidentiality or established institutional responsibilities take priority. The prototype did not compare these alternatives empirically, so the choice would require further assessment against the requirements of a real remittance corridor.

Overall, the prototype demonstrated how a 1:1 reserve ledger can be linked to on-chain transfers

and simulated redemption. It also showed the limits of that demonstration: maintaining an internal accounting invariant and verifying blockchain records are necessary design controls, but they do not independently establish reserve adequacy, secure custody, regulatory compliance or successful fiat redemption.

5.2 Graduate Extension Part C: Policy Brief

5.2.1 Public Stablecoin Rails for Inbound Remittances

Policy question: Should a central bank support the use of public blockchain-based stablecoins for inbound remittances?

Introduction

Inbound remittances provide an important channel through which households receive funds from family members working abroad. However, the cost, speed and accessibility of cross-border transfers vary across corridors and service providers. The World Bank's *Remittance Prices Worldwide* database provides a benchmark for assessing whether alternative payment arrangements deliver measurable improvements in remittance costs (World Bank).

Public blockchains offer an alternative settlement rail through which digital assets can be transferred between wallets without requiring every transfer to pass through the same intermediary chain. However, a blockchain transaction and a completed fiat payment are not equivalent. Antonopoulos (2015) explains how transactions are propagated, validated and recorded through the Bitcoin network. Applied to remittances, this distinction highlights that confirmation of a digital transfer does not, by itself, establish that the recipient has received local currency. The issuer must honour redemption obligations, and the local payment provider must be able to complete the payout. This brief recommends a limited, supervised pilot rather than unrestricted adoption.

Potential Benefits and Principal Risks

Public blockchain rails may support continuous transaction processing, transparent transaction records and programmable transfers. These characteristics could improve traceability and reduce reliance on some intermediaries. Nevertheless, the total cost to the recipient must include network fees, exchange-rate spreads, provider charges and local cash-out costs. A pilot should therefore compare end-to-end costs and payout times with existing services operating in the same remittance corridor.

The principal financial risk is that a stablecoin may lose its intended peg or become difficult to redeem. A reserve balance recorded by an issuer does not, by itself, establish that sufficient liquid assets are

available to meet redemption requests. Weak reserve management, loss of confidence or redemption delays could reduce the value received by beneficiaries. The Committee on Payments and Market Infrastructures and the International Organization of Securities Commissions emphasise the relevance of financial market infrastructure principles to systemically important stablecoin arrangements, including governance, risk management and settlement finality (Committee on Payments and Market Infrastructures and International Organization of Securities Commissions, 2022). Appropriate safeguards should include enforceable redemption rights, independent reserve assurance and transparent disclosures on reserve composition and liquidity.

Operational and compliance risks also require attention. Users may lose access to funds through compromised private keys, fraudulent wallet providers or service outages. Furthermore, public transaction records do not necessarily identify the real-world parties to a transfer. The Financial Action Task Force has highlighted continuing challenges in implementing anti-money laundering and counter-terrorist financing standards for virtual assets and virtual asset service providers (Financial Action Task Force, 2024). Regulated access points should therefore apply customer due diligence, transaction monitoring, sanctions screening and suspicious transaction reporting in accordance with applicable requirements. A cryptographically valid transfer should not be treated as proof of regulatory compliance.

Regulatory and Infrastructure Design

The central bank should assess the complete stablecoin arrangement rather than the blockchain in isolation. The Financial Stability Board recommends comprehensive and risk-proportionate regulation, supervision and oversight of global stablecoin arrangements (Financial Stability Board, 2023). This is particularly important when the issuer, reserve custodian, blockchain infrastructure and remittance provider operate across different jurisdictions.

A pilot should be limited to appropriately authorised issuers and service providers. Before launch, the arrangement should establish legally enforceable redemption into local currency, segregation and independent verification of reserve assets, clear responsibility for custody and operational failures, compliance with applicable AML or CFT requirements, and transparent disclosure of fees and exchange rates. Cooperation between the sending and receiving jurisdictions would also be necessary where responsibilities span national borders.

The design must distinguish transaction confirmation from legal settlement finality and fiat payout. As discussed by Antonopoulos (2015), blockchain transactions are processed through a network of participants and recorded in a shared ledger. However, the legal and operational conditions governing redemption and payout depend on the wider payment arrangement. The pilot should therefore specify when a transfer is considered sufficiently confirmed, when the recipient can access the funds, and how

failed payments, disputes and reversals are handled.

Recommendation and Conclusion

The central bank should permit a controlled pilot only after the legal, prudential and compliance arrangements have been assessed. The pilot should use a clearly specified fiat-referenced stablecoin, regulated on- and off-ramp providers, independently verifiable reserves and explicit redemption obligations. Transaction and exposure limits should be defined in advance, alongside reporting and monitoring requirements.

Evaluation should compare the pilot with existing remittance channels using total customer cost, end-to-end payout time, redemption success, liquidity availability, failed transactions, complaints, fraud incidents and compliance outcomes. World Bank remittance cost data can provide a broader benchmark, but the comparison should focus on services operating in the same corridor (World Bank). Any expansion should depend on observed performance rather than blockchain transparency or transaction speed alone.

Public stablecoin rails may improve some aspects of cross-border value transfer, but they do not eliminate counterparty, liquidity, legal or compliance risks. A supervised pilot would allow the central bank to assess these trade-offs while limiting exposure. If enforceable redemption, reliable reserves, effective oversight or measurable customer benefits cannot be demonstrated, existing regulated payment infrastructure or a permissioned arrangement would be more appropriate.

6 RESULTS AND DEMO EVIDENCE

6.1 Results and Demo Evidence

The prototype was tested by depositing 5,000 units of local currency, transferring 3500 mZAR to the recipient, and then converting the 3500 mZAR back into local currency. The tests showed that the main steps worked as expected.

6.1.1 Wallet Creation and On-Ramp

Three wallets were created for the sender, recipient, and reserve agent. Each wallet had its own address. The sender then deposited 5,000 units of local currency and received 5,000 mZAR. After the deposit, both the reserve and the total amount of mZAR in circulation were 5,000. This means that the amount held in reserve matched the amount of mZAR issued.

6.1.2 Transfer and Mining

The sender transferred 3500 mZAR to the recipient. The transaction was signed and added to the mempool before being included in block 1. After mining, the sender had 1500 mZAR left, and the recipient had 3500 mZAR. The reserve remained at 5,000 because transferring money between wallets does not change the total amount of mZAR in circulation.

The transaction ID and block hash were recorded as follows:

- Transaction ID: `076f1a7a26bab0bd23f5c97b723ccbb1e6726a1f332f34b0c04a97b1be9b88f0`
- Payment block: 1
- Payment block hash: `00c7d903910c0059...`
- Amount transferred: 3500 mZAR

The transaction ID identifies the payment, while the block hash identifies the block containing it.

6.1.3 Confirmation and Verification

The payment initially had one confirmation. Another block was then mined to show how additional blocks increase the confirmation depth. The payment reached two confirmations in this test. The `verify_chain()` function also checked whether the blocks and their hashes were linked correctly. The test returned `chain_ok = True`, meaning that the chain passed the verification check.

6.1.4 Off-Ramp and Reserve Accounting

The recipient then converted the 3500 mZAR back into local currency using the simulated off-ramp. A receipt ID was created to record the payment.

The final balances were:

Item	Final amount
Sender balance	1500 mZAR
Recipient balance	0 mZAR
Reserve balance	1500 local currency units
Circulating supply	1500 mZAR

The reserve and circulating supply both ended at 1500. This shows that the system reduced the reserve and the amount of mZAR in circulation when the recipient redeemed the funds. The off-ramp also generated a receipt ID. In this test, the receipt ID was `910ba32d588245b9`. The ID may be different when the program is run again.

6.1.5 Testing Invalid Transactions

Two invalid transactions were tested. First, the amount was changed after the transaction had been signed. The system rejected it because the signature was no longer valid. Second, the sender tried to send 5,000 mZAR even though only 5,000 mZAR was available. The transaction was rejected because the sender did not have enough funds. In both tests, the invalid transactions were not added to the mempool. These tests showed that the system could reject transactions with invalid signatures or insufficient funds.

6.1.6 Blockchain Explorer

The blockchain explorer displayed information about each block, including its index, hash, and transaction details. It also showed the transaction ID, sender's address, recipient's address, and amount transferred. The chain verification test passed, meaning that the blocks passed the system's checks for their hashes and links.

6.1.7 Payment API Testing

The Flask API was tested using Flask's test client. The tests covered depositing money, transferring mZAR, mining blocks, checking balances, withdrawing money, and viewing the blockchain. The API successfully processed the valid transfer and rejected the tampered transaction. The full remittance test also passed. At the end, the reserve balance was 1500, and both the reserve check and blockchain verification passed. The output displayed `Flask test_client remittance API OK`, confirming that the API test was successful.

6.1.8 Overall Results

Overall, the tests showed that the main steps of the remittance process worked. The sender could deposit local currency, transfer mZAR to the recipient, and convert it back into local currency. The system also recorded transaction details, created a receipt, checked the blockchain, and rejected the invalid transactions that were tested. However, this was only a prototype. The money and mZAR were simulated, so the system did not make real bank payments or use a live stablecoin network.

7 LIMITATIONS AND FUTURE WORK

7.1 Limitations and Future Work

The prototype was able to demonstrate the main remittance process, but there are some limitations because the system was designed as a classroom prototype rather than a real payment system.

7.1.1 Limitations

There are a few limitations to the prototype. First, the wallets and digital signatures are simplified. We used SHA-256 values to represent keys and addresses. This was enough to demonstrate signing and verification, but it is not the same as using real cryptocurrency wallets, which require proper private key generation and protection. The prototype also does not have a complete system for protecting private keys from being lost or stolen. Second, the mZAR stablecoin and its reserves are simulated. We assumed that one unit of local currency could be converted into one mZAR. The reserve ledger keeps track of the local currency and the amount of mZAR in circulation, and the tests showed that these amounts remained equal. However, the system is not connected to a real bank account, so it does not handle actual money or confirm that real funds are available. A real system would need proper reserve management and regular checks to ensure that enough money is held to support the tokens issued.

Third, the mining process uses a proof-of-work difficulty of 2 so that blocks can be mined quickly during the demonstration. This helps explain how mining works, but it does not provide the same level of security as a real blockchain network. The confirmation process is also simplified. The payment was included in block 1 and initially had one confirmation. Mining additional blocks increased the confirmation depth to two in the separate test. More confirmations can make a transaction harder to reverse, but they do not guarantee that it can never be reversed. A real system would need to consider the blockchain's security and decide how many confirmations are appropriate for each payment.

The prototype also does not include important financial controls such as Know Your Customer (KYC) checks, Anti-Money Laundering (AML) checks, fraud detection, transaction limits, user authentication, and regulatory reporting. These would be needed to help prevent fraud, protect users, and meet legal requirements if the system were used for real remittance payments. Finally, a public blockchain

may not always be the best option for a remittance service. It can involve transaction fees, delays, and privacy concerns. If a trusted organisation manages the payments, a conventional database or a permissioned blockchain may be simpler and more suitable. The best option would depend on the cost, security, privacy, and level of trust required.

Overall, the prototype demonstrates the main steps of a remittance payment, but further development would be needed before it could be used to transfer real money safely and reliably.

7.1.2 Future Work

There are several ways in which the prototype could be improved in the future. The wallet system could be replaced with proper public-key cryptography and safer storage of private keys. The current on-ramp and off-ramp are also simulated. A future version could connect to a real bank or payment API so that actual money movement is represented. The blockchain could also be made more realistic by using a stronger consensus method, a more realistic mining difficulty, and multiple nodes. Security features such as user authentication, KYC, AML checks, fraud detection, and transaction limits could also be added. Another improvement would be to create one API that handles the full payment process and provides the user with the transaction and block information. Overall, the prototype demonstrates the basic remittance process from local currency to mZAR, the transfer and recording of mZAR on the blockchain, and the conversion back to local currency. Future work would mainly focus on making these simplified components more secure, realistic, and suitable for a real financial system.

8 DISCLOSURES

8.1 Contribution statement

1. Keitumetse - Report :Executive summary and Architecture plus Graduate Extensions Part B (Mastering Ethereum)

2. Siphosethu - Results and Limitations

3. Masixole - Financial design plus Graduate Extensions Part C (Policy brief)(Part C)

4. Joelle - Implementation - key design decisions; PoW difficulty used for demo; API surface

8.2 AI Discloure

AI was used to assist with the development and debugging of the code, and improving the clarity of the report. All outputs were reviewed and adapted to reflect our understanding of the work.

8.3 Github link

Be careful with the dash after py. If it fails when clicking please try adding dash after py.

https://github.com/max035/Block-Chain-Assignment-9

BIBLIOGRAPHY

Andreas M. Antonopoulos. *Mastering Bitcoin: Unlocking Digital Cryptocurrencies*. O'Reilly Media, 1 edition, 2015.

Andreas M. Antonopoulos and Gavin Wood. *Mastering Ethereum: Building Smart Contracts and DApps*. O'Reilly Media, 1 edition, 2018.

Committee on Payments and Market Infrastructures and International Organization of Securities Commissions. Application of the principles for financial market infrastructures to stablecoin arrangements, 2022. URL https://www.bis.org/cpmi/publ/d206.htm.

Financial Action Task Force. Virtual assets: Targeted update on implementation of the fatf standards on vas and vasps, 2024. URL https://www.fatf-gafi.org/en/publications/Fatfrecommendations/targeted-update-virtual-assets-vasps-2024.html.

Financial Stability Board. High-level recommendations for the regulation, supervision and oversight of global stablecoin arrangements: Final report, 2023. URL https://www.fsb.org/2023/07/high-level-recommendations-for-the-regulation-supervision-and-oversight-of-glo

World Bank. Remittance prices worldwide. World Bank Data Catalog. URL https://datacatalog.worldbank.org/search/dataset/0037898/remittance-prices-worldwide.